```
template<typename Key,typename Value,typename Hash=std::hash<Key> >
class threadsafe_lookup_table
{
private:
    class bucket_type
    {
    private:
        typedef std::pair<Key,Value> bucket_value;
        typedef std::list<bucket_value> bucket_data;
        typedef typename bucket_data::iterator bucket_iterator;

        bucket_data data;
        mutable boost::shared_mutex mutex;     <-(1)

        bucket_iterator find_entry_for(Key const& key) const    <-(2)
        {
            return std::find_if(data.begin(),data.end(),
                                [&](bucket_value const& item)
                                {return item.first==key;});
        }

    public:
        Value value_for(Key const& key,Value const& default_value) const
        {
            boost::shared_lock<boost::shared_mutex> lock(mutex);     <-(3)
            bucket_iterator const found_entry=find_entry_for(key);
            return (found_entry==data.end())?
                default_value:found_entry->second;
        }
```